

urlAPI 项目

曾浩正，童济州

2026 年 6 月 11 日



目录

- ① 项目定位
- ② 项目分层架构
- ③ 分发方式
- ④ 功能展示
- ⑤ 面向人群
- ⑥ 特性设计
- ⑦ 开源合规性

项目定位

一句话概括

urlAPI 是一个 Go + Vue 的 URL 化 API 服务：把文本生成、图片生成、随机图片、网页缩略图和 OpenAI 兼容模型接口封装成可直接调用的 Web 服务，并提供后台管理能力。

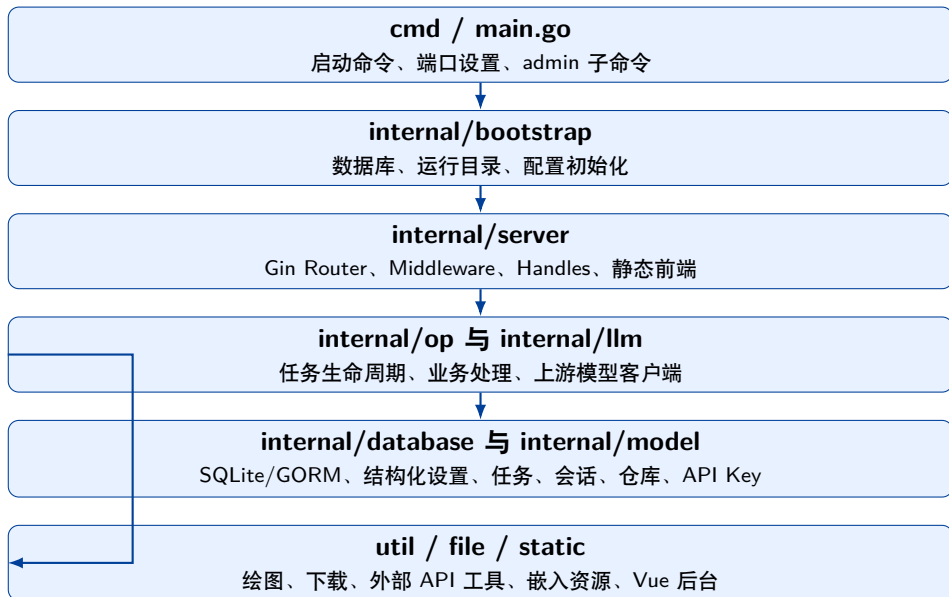
主要能力

- 文字生成并渲染为 PNG 图片
- 图片生成并通过下载接口中转
- GitHub/Gitee 仓库随机图片
- Bilibili、YouTube、arXiv、GitHub 等网页信息卡片
- OpenAI-compatible /v1 接口

管理能力

- 后台登录、任务查看与筛选
- 供应商、模型、参数和 Prompt 配置
- Referer、IP、Prompt、功能开关安全策略
- API Key 创建、配额、IP 白名单和启停

项目分层架构



路由与请求流

公开 URL API

- GET /txt: 文本生成图片
- GET /img: AI 图片生成
- GET /rand: 随机图片
- GET /web: 网页缩略图
- GET /download: 临时图片下载

后台与模型接口

- POST /session: 登录、任务、仓库、配置
- /admin/apikeys: API Key 管理
- /v1/chat/completions: 聊天补全
- /v1/embeddings: 向量接口
- /v1/models: 模型列表

典型请求流

Router 装载安全中间件 → Handler 读取参数 → op 执行业务与任务缓存 → database 记录任务 → JSON 返回或 302 跳转到结果 URL。

分发与部署方式

方式	说明
单体 Docker	0w0w0/urlapi:latest, Go 后端内嵌 Vue 前端, 默认服务端口 2233, 宿主机目录挂载到 /app/assets 保存 SQLite 与临时资源。
前端独立 Docker	0w0w0/urlapi-frontend:latest, Nginx 托管前端, 通过 BACKEND_URL 反代后端公开接口。
ARM 镜像	README 约定使用 0w0w0/urlapi-arm。
二进制文件	GitHub Actions 构建多平台产物, 可从 Actions 或 Release 获取。
systemd 服务	用 ExecStart=/root/urlAPI/urlAPI port 2233 注册为系统服务。

启动参数

支持 port、admin repwd、admin clear、admin logout、admin clear_ip_restriction, 用于端口设置、重置密码、清空任务和会话。

访问方式一：URL + Referer 白名单

面向博客、页面和简单自动化：把结果当作图片或 JSON 使用。

```
curl -L "https://api.example.com/txt?prompt=poem&format=json"
curl -L "https://api.example.com/img?prompt=cat&api=openai&size=1024x1024"
curl -L "https://api.example.com/rand?api=github&user=me&repo=images"
curl -L "https://api.example.com/web?img=https://github.com/owner/repo"
```

- 默认非 JSON 请求返回 302，浏览器或 `curl -L` 会跟随到图片地址。
- `format=json` 返回结构化结果，便于脚本调试。
- 公开接口经过 Referer 白名单、频率限制、功能开关、Prompt 通配符等检查。

访问方式二： curl + API Key

面向开发者：通过 OpenAI 兼容 /v1 接口接入。

```
curl https://api.example.com/v1/chat/completions \  
  -H "Authorization: Bearer sk-..." \  
  -H "X-Provider: deepseek" \  
  -H "Content-Type: application/json" \  
  -d '{"model": "deepseek-chat", "messages": [{"role": "user", "content": "Hi"}]}'
```

- API Key 可从 Authorization: Bearer、X-API-Key 或 api_key 查询参数读取。
- 支持 Key 启停、角色、日/月配额、过期时间和 IP 白名单。
- X-Provider 可切换供应商；不传时使用后台默认文字供应商。

功能： 供应商、模型和参数调整

供应商配置

- OpenAI、DeepSeek、Alibaba、OtherAPI
- Anthropic、Gemini、Azure、Moonshot
- API Key、endpoint、启用状态
- 文本、总结、图片、Embedding 默认模型

模型参数

- temperature
- max tokens
- top p
- presence penalty
- frequency penalty
- custom headers

设计目标

上游模型服务以 ProviderConfig 形式统一接入，业务层只关心“使用哪个 provider、哪个 model、哪个 endpoint”。

功能：README 中的四类公开服务

功能	参数与行为
文字生成	prompt 必填; api/model/more/format 可选; 可使用 laugh、poem、sentence 或自定义 Prompt。
图片生成 随机图片	prompt 必填; api/size/format/more 可选; 支持 OpenAI 与 Alibaba 图片能力。 user/repo 必填; api 可选 github/gitee; 从后台维护的仓库图片列表中随机取图。
网页缩略图	img 参数传入页面 URL; 支持视频、论文、新闻、代码仓库等信息卡片绘制。

功能：结合代码发现的更多能力

- **OpenAI 兼容代理**：实现 `/v1/chat/completions`、`/v1/embeddings`、`/v1/models`，可作为模型网关使用。
- **流式输出**：聊天补全支持 `stream=true`，以 SSE 格式返回 `data: ...` 与 `[DONE]`。
- **API Key 治理**：Key 哈希入库，支持配额、启停、过期、IP 限制和使用量统计。
- **后台工作台**：组合 Host、Operation、Query 参数生成 URL，并预览调用结果。
- **结构化设置 API**：`fetchSettings/editSettings` 使用具名 `setting_body`，避免前端直接依赖数组下标。
- **资源嵌入**：Vue dist、字体、图标、Logo 和默认配置随 Go 二进制嵌入。

功能：访问数据统计与用量追踪

任务与用量统计

- Task 表中记录每次生成/下载请求的 IP、Referer、状态、API、模型等字段。
- 后台可筛选时间范围、任务类型、状态、来源域名，用于生成访问报表。
- APIKeyUsage 日志记录每次 API Key 调用的端点、IP、状态码和耗时。
- GetUsageStats 支持按 API Key 和起止时间查询调用量。

配额与并发统计

- API Key 记录 usage_day 和 usage_month，实时追踪日/月使用量。
- CheckQuota 校验调用是否超出日/月配额上限。
- TaskCounter 按上游 API 统计当前并发任务数，防止打满。
- ResetDailyUsage/ResetMonthlyUsage 自动重置过期日/月计数。

统计维度汇总

项目支持按时间范围、任务类型、成功/失败状态、来源 IP、Referer 域名、API 供应商、API Key 等维度统计访问量和生成量，并提供配额上限告警。

开发者

- 需要统一接入多个大模型供应商
- 需要 OpenAI-compatible API Key 网关
- 需要可追踪的任务记录和配额控制
- 需要用 Docker 或二进制快速私有部署

博客作者

- 需要把动态内容嵌入文章或页面
- 需要随机图、AI 配图、文字卡片
- 需要通过 URL 直接调用，无需写复杂前端
- 需要 Referer 白名单防止公开接口被滥用

特性设计：执行队列

核心数据结构

TaskQueue 以 Type + Size + Target + API 作为去重键；TaskCounter 按 API 统计并发任务数。

- 同一任务正在运行时，后续请求等待结果，避免重复调用上游 API。
- 成功任务在缓存时间内复用结果，当前任务标记 Temp=Yes。
- 过期或失败任务会清理旧临时图片，重新执行。
- 每个 API 的并发执行数受控，避免上游接口或本机资源被打满。
- 任务结束后会校验 PNG 文件有效性，异常时返回空图并记录失败。

特性设计：图片缓存能力

缓存目标

对文字生成图、AI 生成图和网页缩略图进行结果复用，减少重复调用上游 API、重复绘图和重复下载。

- 缓存键复用 TaskQueueFilter：按任务类型、尺寸、目标内容和 API 供应商区分结果。
- 生成成功后图片保存到 `assets/img/<uuid>.png`，对外统一返回 `/download?img=<uuid>`。
- 缓存过期时间来自后台配置：文字、图片、网页缩略图分别有独立 `cache_minutes`。
- 命中缓存时不重新生成图片，而是复制历史任务结果并标记 `Temp=Yes`，仍可按需写入任务记录。
- 缓存过期、生成失败或 PNG 校验失败时会丢弃旧结果，重新执行并在失败时返回空图或 fallback。
- 临时图片目录在服务启动时重建，因此缓存是运行期缓存；持久化任务记录保留，但旧下载 URL 可能随重启失效。

特性设计：流控与风控

流控

- 公开 URL API 按 Type + IP 维护访问频率窗口。
- 0.25 秒内超过 10 次同类请求会被拒绝，避免脚本刷接口。
- API Key 支持日配额、月配额和使用量统计，超额返回 `rate_limit_error`。
- TaskCounter 按上游 API 统计并发，防止模型接口和本机资源被打满。

风控

- 功能开关控制文字、图片、随机图、网页缩略图是否开放。
- Prompt 支持启用列表和通配符白名单，限制可生成内容范围。
- API 参数限制在受支持供应商集合内，阻断非法 provider 输入。
- 可选 HMAC 签名校验，使用时间戳降低重放风险。

特性设计：白名单机制

白名单	项目实施与作用
Referer 白名单	<code>Security.AllowedReferers</code> 校验来源域名，公开图片类接口必须来自允许域名，保护博客嵌入场景。
API Key IP 白名单	每个 Key 可配置 <code>allowed_ips</code> ，请求 IP 不匹配时拒绝，* 表示放行。
Web 目标白名单	<code>Web.AllowedHosts</code> 限定网页缩略图可解析的站点，避免服务被当作任意 URL 抓取器。
Prompt 白名单	文字 Prompt 使用启用列表和 <code>accepted_prompt_glob</code> ；图片 Prompt 使用 <code>accepted_prompt_glob</code> 。
任务记录例外	<code>Task.ExceptDomains/ExceptInfos</code> 命中后跳过数据库记录，减少可信来源或特定错误信息的噪声。

特性设计：上游 API 对接

统一抽象

- internal/llm 封装 provider client
- ProviderConfig 描述 endpoint、模型、参数
- 上层 Handler 不直接拼接各家请求细节

兼容与扩展

- OpenAI Chat/Embedding/Models 协议
- DeepSeek 与 OtherAPI 走兼容接口
- Alibaba 图片生成采用异步任务轮询
- Anthropic/Gemini/Azure/Moonshot 可配置接入

失败处理

外部调用失败会落入任务记录，并按接口类型返回 JSON 错误、fallback 图片或 OpenAI 兼容错误结构。

特性设计：数据库版本匹配与更新

- 旧版本使用 `settings.name + settings.value`(JSON 字符串数组)，业务和前端依赖魔法下标。
- 新设计拆出 `providers`、`service_configs`、`prompts`、`config_list_items`、`app_settings` 等结构化表。
- 启动时通过 `SettingsStore` 加载结构化配置，并用读写锁提供线程安全快照。
- 保存设置时先构建 V2 rows，在事务中替换相关表，再替换内存快照。
- 新 `fetchSettings/editSettings` 返回具名 DTO，敏感字段不明文回显，空值保留旧密钥。
- 迁移文档要求保留旧表一段时间，支持从旧数组配置安全迁移并避免破坏现有行为。

核心结论

urlAPI 当前仓库已包含 GPL-3.0 许可证；本次补充了 NOTICE 与 OPEN_SOURCE_COMPLIANCE.md，把分发、依赖、SBOM、SCA 和 AI 服务条款审查要求落到项目文件中。

- 项目主许可证为 textbfGPL-3.0，属于高风险强 Copyleft：分发修改版或衍生作品时需要提供对应源代码。
- 后端依赖清单来自 go.mod/go.sum，前端依赖清单来自 static/package.json/package-lock.json。
- 尚未发现完整第三方许可证汇总文件，因此 NOTICE 要求发布前按精确版本生成第三方 notices。
- 项目对接多家模型与内容服务，除代码 License 外，还需要审查 API 服务条款、模型社区许可和生成内容使用边界。

开源合规性：技术管控

- **SBOM**：以 `go.mod/go.sum` 和 `package-lock.json` 生成为准，记录直接依赖和间接依赖的名称、版本、许可证。
- **SCA 扫描**：在 CI/CD 中接入 FOSSology、Snyk、SonarQube、Black Duck 等工具，识别 GPL/AGPL、未知许可证和复制粘贴代码。
- **交付物合规**：Docker 镜像、二进制和前端 bundle 应附带 LICENSE、NOTICE、第三方 notices 和源码获取路径。
- **修改声明**：如修改第三方文件，需要保留原版权并标记修改；Apache-2.0 等许可证对 NOTICE 有明确要求。
- **证据链**：保留 Git 提交、Release tag、构建日志、扫描报告和依赖锁文件，支撑后续审计。

组织治理

- 建立许可证准入白名单、灰名单和黑名单。
- 高风险许可证、模型服务和商用分发需法务/架构审查。
- 发布前执行合规 checklist，输出可追溯扫描结果。

AI 合规

- 检查 Llama、Qwen 等社区许可证的 MAU、用途和竞争模型限制。
- 记录训练、微调、提示词和生成资源的数据来源与变更日志。
- 避免将隐私、商业秘密或未授权数据发送给上游模型供应商。

项目价值

urlAPI 将“模型能力、图片能力、网页摘要能力、后台治理能力”组合为一个可私有部署的 URL/API 网关，既能服务博客内容生产，也能作为开发者模型接入层。

- 架构上已从早期平铺目录迁移到 `cmd + internal + util + static` 的分层结构。
- 功能上覆盖公开 URL API、OpenAI 兼容接口、后台配置、任务记录和 API Key 管理。
- 设计上重点解决执行队列、图片缓存、流控风控、白名单、上游 API 适配、配置结构化和数据库迁移问题。
- 合规上已明确 GPL-3.0 分发义务，并补充 NOTICE、SBOM/SCA、交付物 notices 和 AI 服务条款审查要求。